In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import datetime
import missingno as msno
import matplotlib
import matplotlib.pylab as pylab

%matplotlib inline
matplotlib.style.use('ggplot')
sns.set_style('white')
pylab.rcParams['figure.figsize'] = 8,6

import math
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from math import sqrt
from sklearn.linear_model import BayesianRidge
from sklearn.linear_model import LassoLars
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import ElasticNet
```

In [2]:
```
rossman_df=pd.read_csv('C://BabanDellLaptop//NMIMS//Final_Project//train.csv', low_
```

In [3]:
```
rossman_df.head()
```

Out[3]:

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHolida
0	1	5	2015-07-31	5263	555	1	1	0	
1	2	5	2015-07-31	6064	625	1	1	0	
2	3	5	2015-07-31	8314	821	1	1	0	
3	4	5	2015-07-31	13995	1498	1	1	0	
4	5	5	2015-07-31	4822	559	1	1	0	

In [4]:
```
store_df=pd.read_csv('C://BabanDellLaptop//NMIMS//Final_Project//store.csv', low_me
```

In [5]:
```
rossman_df.tail()
```

Out[5]:

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	Schoo
1017204	1111	2	2013-01-01	0	0	0	0	a	
1017205	1112	2	2013-01-01	0	0	0	0	a	
1017206	1113	2	2013-01-01	0	0	0	0	a	
1017207	1114	2	2013-01-01	0	0	0	0	a	
1017208	1115	2	2013-01-01	0	0	0	0	a	

In [6]:
```
rossman_df.shape
```

Out[6]:
```
(1017209, 9)
```

In [7]:
```
rossman_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1017209 entries, 0 to 1017208
Data columns (total 9 columns):
 #   Column         Non-Null Count    Dtype 
---  ------         --------------    ----- 
 0   Store          1017209 non-null  int64 
 1   DayOfWeek      1017209 non-null  int64 
 2   Date           1017209 non-null  object
 3   Sales          1017209 non-null  int64 
 4   Customers      1017209 non-null  int64 
 5   Open           1017209 non-null  int64 
 6   Promo          1017209 non-null  int64 
 7   StateHoliday   1017209 non-null  object
 8   SchoolHoliday  1017209 non-null  int64 
dtypes: int64(7), object(2)
memory usage: 69.8+ MB
```

In [8]:
```
rossman_df.describe()
```

Out[8]:

	Store	DayOfWeek	Sales	Customers	Open	Prom
count	1.017209e+06	1.017209e+06	1.017209e+06	1.017209e+06	1.017209e+06	1.017209e+0
mean	5.584297e+02	3.998341e+00	5.773819e+03	6.331459e+02	8.301067e-01	3.815145e-0
std	3.219087e+02	1.997391e+00	3.849926e+03	4.644117e+02	3.755392e-01	4.857586e-0
min	1.000000e+00	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+0
25%	2.800000e+02	2.000000e+00	3.727000e+03	4.050000e+02	1.000000e+00	0.000000e+0
50%	5.580000e+02	4.000000e+00	5.744000e+03	6.090000e+02	1.000000e+00	0.000000e+0
75%	8.380000e+02	6.000000e+00	7.856000e+03	8.370000e+02	1.000000e+00	1.000000e+0
max	1.115000e+03	7.000000e+00	4.155100e+04	7.388000e+03	1.000000e+00	1.000000e+0

In [9]:
```
# extract year, month, day and week of year from "Date"

rossman_df['Date']=pd.to_datetime(rossman_df['Date'])
rossman_df['Year'] = rossman_df['Date'].apply(lambda x: x.year)
rossman_df['Month'] = rossman_df['Date'].apply(lambda x: x.month)
rossman_df['Day'] = rossman_df['Date'].apply(lambda x: x.day)
rossman_df['WeekOfYear'] = rossman_df['Date'].apply(lambda x: x.weekofyear)
```

In [10]:
```
rossman_df.sort_values(by=['Date','Store'],inplace=True,ascending=[False,True])
rossman_df.head(2)
```

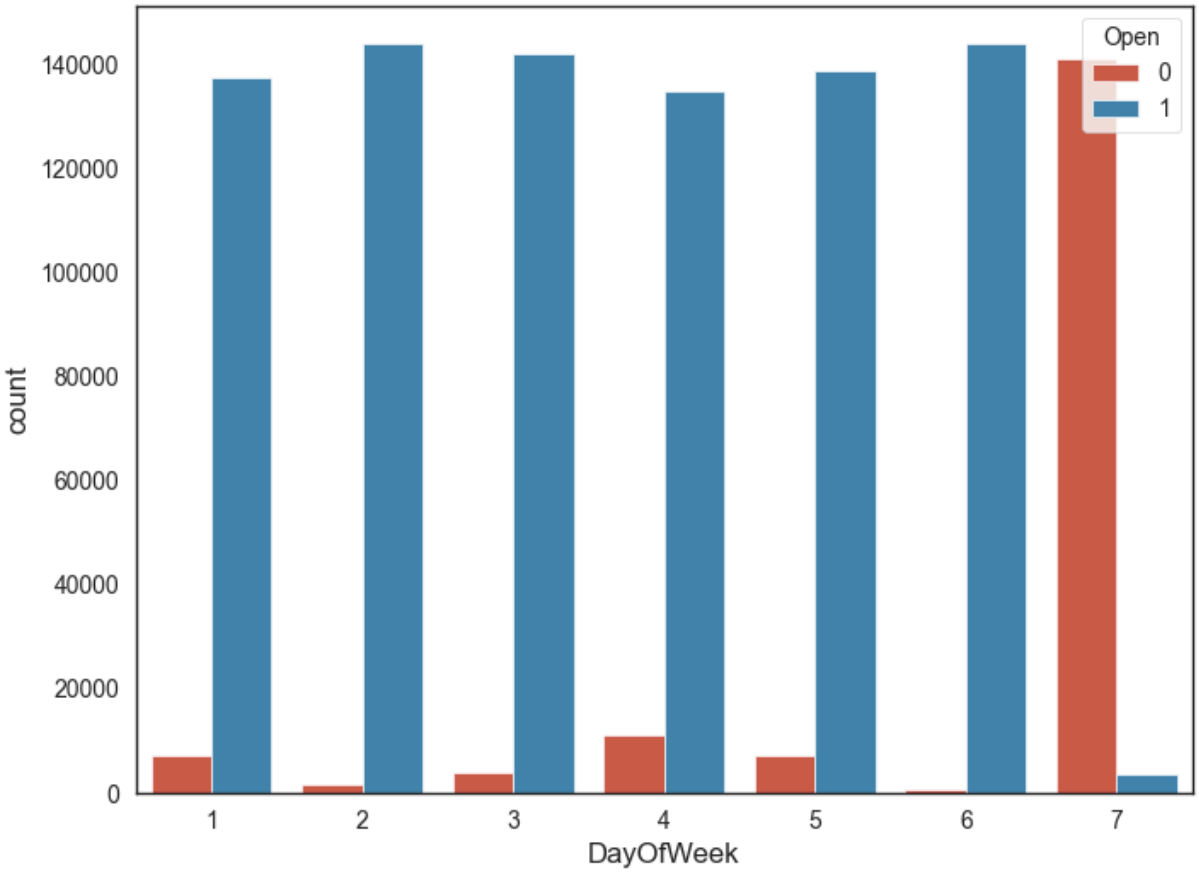
Out[10]:

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHolida
0	1	5	2015-07-31	5263	555	1	1	0	
1	2	5	2015-07-31	6064	625	1	1	0	

Exploratory Data Analysis

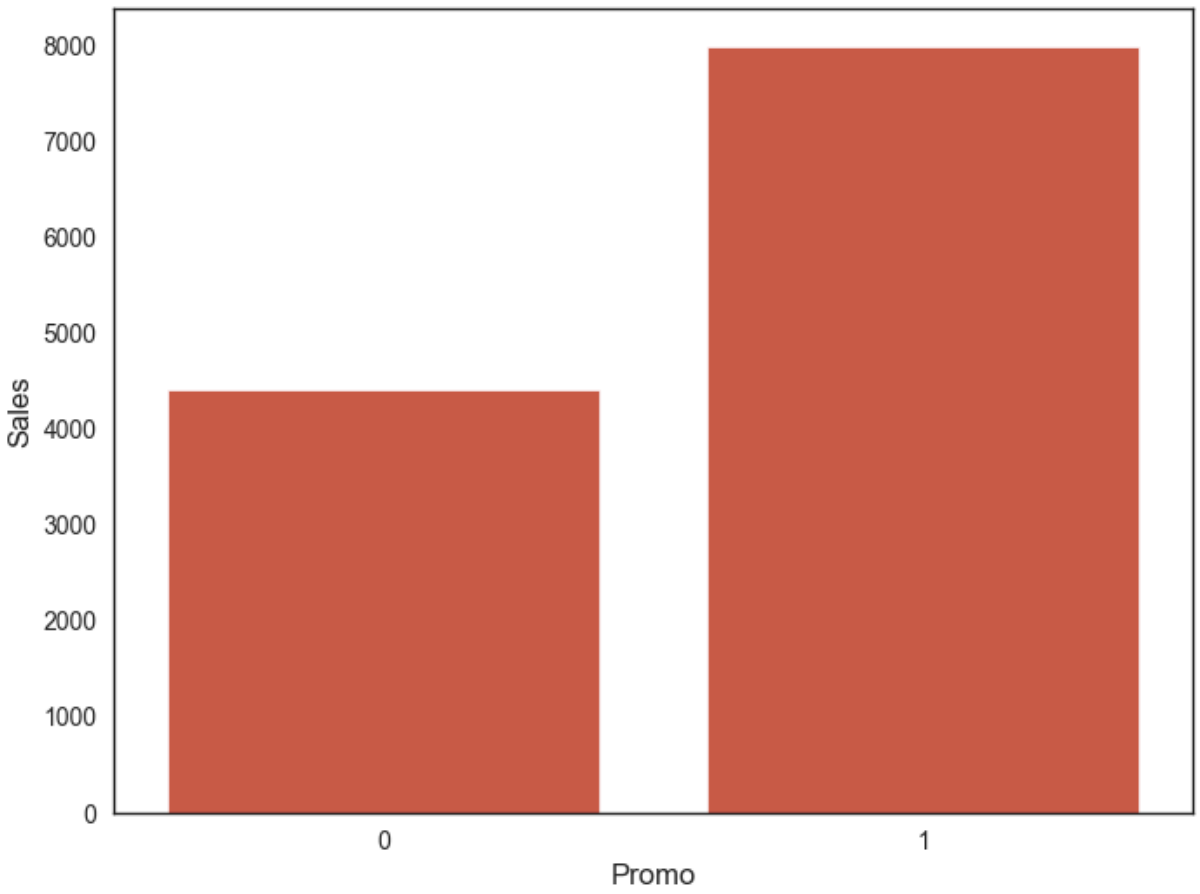
In [11]:
```
sns.countplot(x='DayOfWeek',hue='Open',data=rossman_df)
```

Out[11]:
```
<Axes: xlabel='DayOfWeek', ylabel='count'>
```



In [12]:
```
#Impact of promo on sales
Promo_sales = pd.DataFrame(rossman_df.groupby('Promo').agg({'Sales':'mean'}))
sns.barplot(x=Promo_sales.index, y = Promo_sales['Sales'])
```

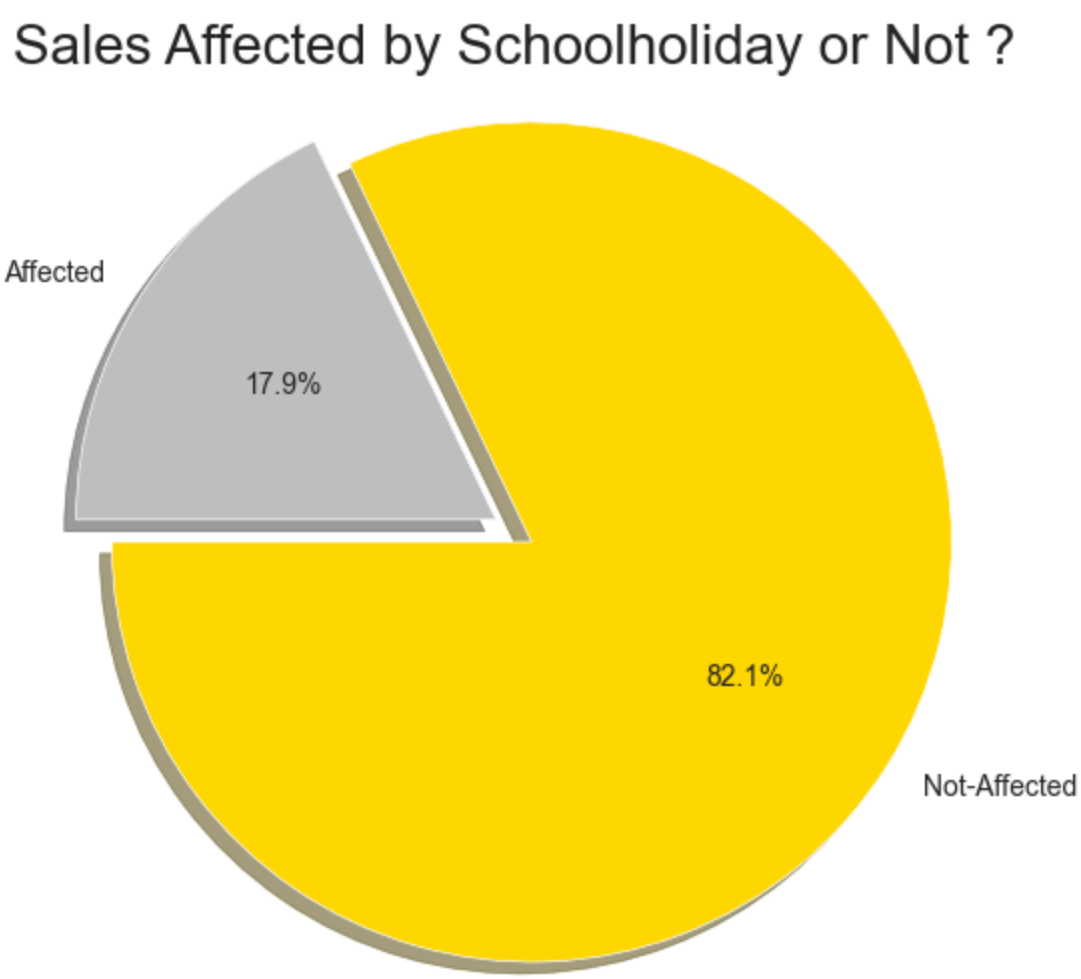
Out[12]:
```
<Axes: xlabel='Promo', ylabel='Sales'>
```



In [13]:
```
# Value Counts of SchoolHoliday Column
rossman_df.SchoolHoliday.value_counts()
```

Out[13]:
```
SchoolHoliday
0    835488
1    181721
Name: count, dtype: int64
```

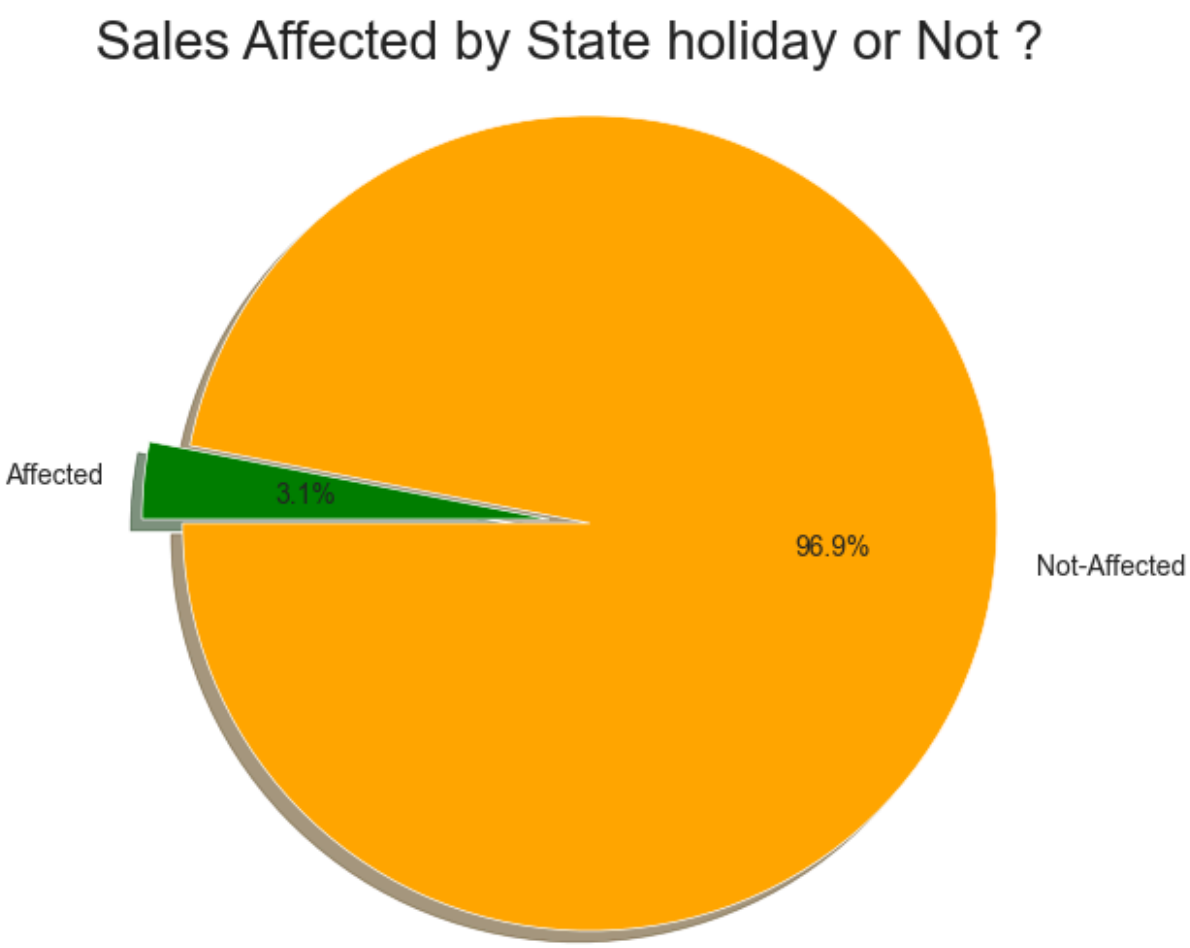
In [14]:
```
labels = 'Not-Affected' , 'Affected'
sizes = rossman_df.SchoolHoliday.value_counts()
colors = ['gold', 'silver']
explode = (0.1, 0.0)
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
        autopct='%1.1f%%', shadow=True, startangle=180)
plt.axis('equal')
plt.title("Sales Affected by Schoolholiday or Not ?",fontsize=20)
plt.plot()
fig=plt.gcf()
fig.set_size_inches(6,6)
plt.show()
```



In [15]:
```
#distribution of sales
#fig, ax = plt.subplots()
#fig.set_size_inches(11, 7)
#sns.histplot(rossman_df['Sales'], kde = False,bins=40);
```

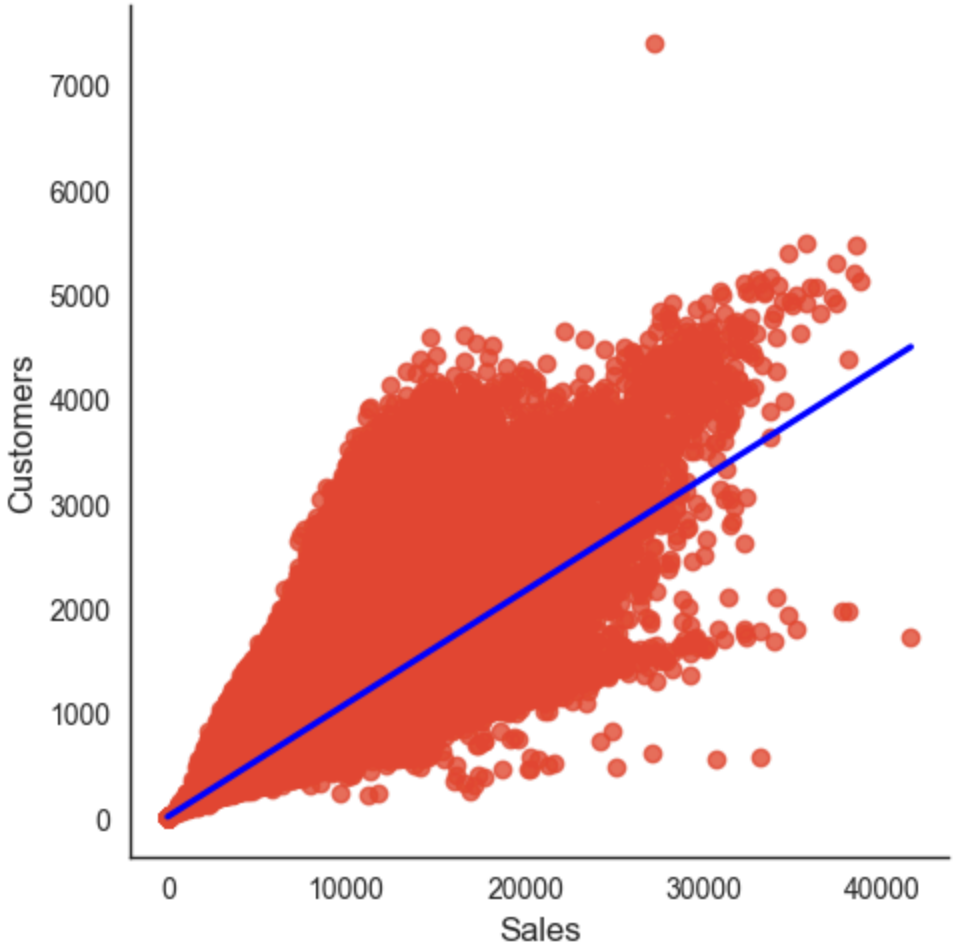
In [16]:
```
rossman_df["StateHoliday"] = rossman_df["StateHoliday"].map({0: 0, "0": 0, "a": 1,
```

In [17]:
```
labels = 'Not-Affected' , 'Affected'
sizes = rossman_df.StateHoliday.value_counts()
colors = ['orange','green']
explode = (0.1, 0.0)
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
        autopct='%1.1f%%', shadow=True, startangle=180)
plt.axis('equal')
plt.title("Sales Affected by State holiday or Not ?",fontsize=20)
plt.plot()
fig=plt.gcf()
fig.set_size_inches(6,6)
plt.show()
```



In [18]:
```
rossman_df.drop('StateHoliday',inplace=True,axis=1)
```

In [19]:
```
#Linear relation between sales and customers
sns.lmplot(x= 'Sales' , y ='Customers',data=rossman_df, palette='seismic', height=5
```



Analysis of Store Data

In [20]:
```
store_df.head(5)
```

Out[20]:

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	Comp
0	1	c	a	1270.0	9.0	
1	2	a	a	570.0	11.0	
2	3	a	a	14130.0	12.0	
3	4	c	c	620.0	9.0	
4	5	a	a	29910.0	4.0	

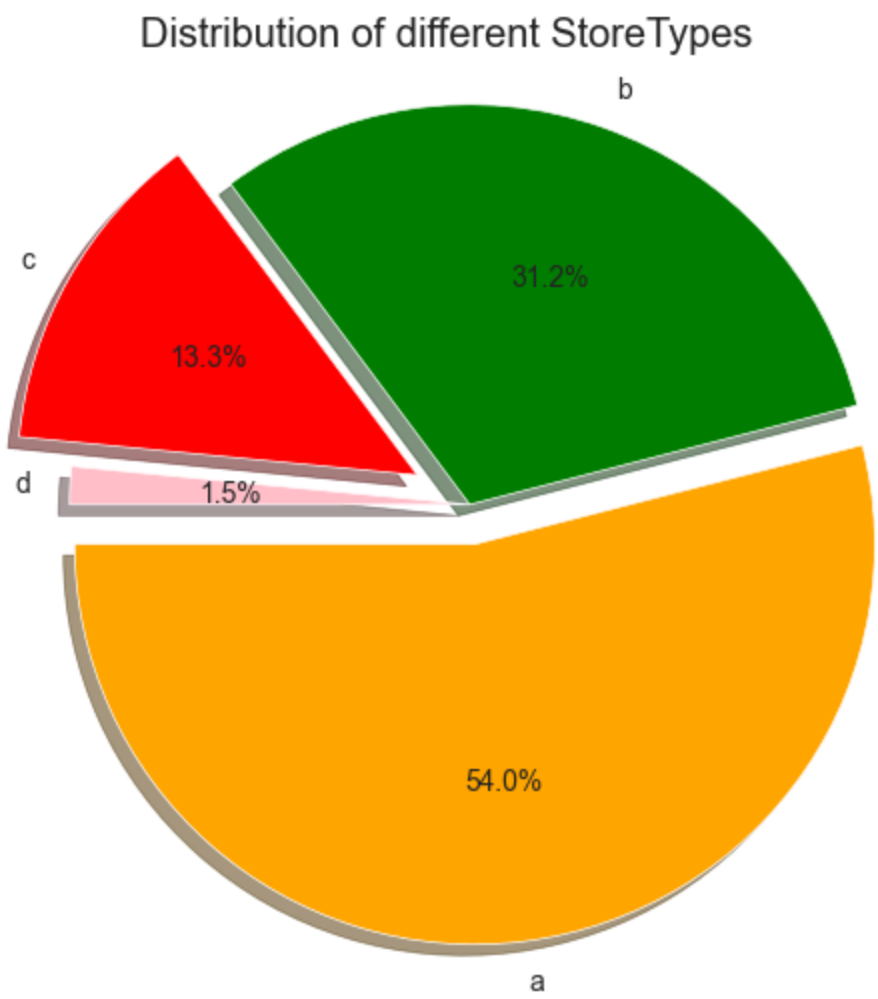
In [21]:
```
store_df.tail()
```

Out[21]:

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	C
1110	1111	a	a	1900.0	6.0	
1111	1112	c	c	1880.0	4.0	
1112	1113	a	c	9260.0	NaN	
1113	1114	a	c	870.0	NaN	
1114	1115	d	c	5350.0	NaN	

Distribution of Type of STores

In [22]:
```
labels = 'a' , 'b' , 'c' , 'd'
sizes = store_df.StoreType.value_counts()
colors = ['orange', 'green' , 'red' , 'pink']
explode = (0.1, 0.0 , 0.15 , 0.0)
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
        autopct='%1.1f%%', shadow=True, startangle=180)
plt.axis('equal')
plt.title("Distribution of different StoreTypes")
plt.plot()
fig=plt.gcf()
fig.set_size_inches(6,6)
plt.show()
```

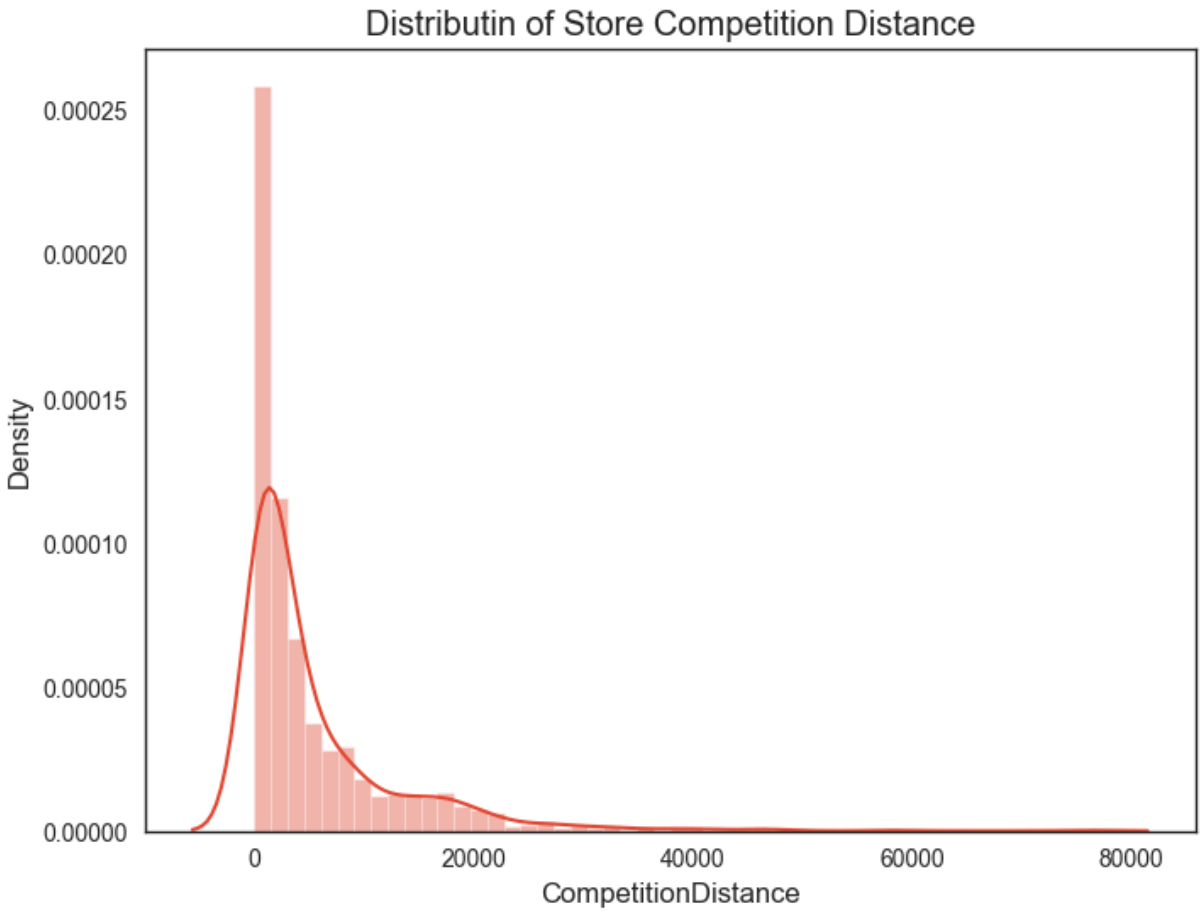


In [23]:
```
# remove features
store_df = store_df.drop(['CompetitionOpenSinceMonth', 'CompetitionOpenSinceYear','
                          'Promo2SinceYear', 'PromoInterval'], axis=1)
```

In [24]:
```
sns.distplot(store_df.CompetitionDistance.dropna())
plt.title("Distributin of Store Competition Distance")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_13208\983448205.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(store_df.CompetitionDistance.dropna())
```

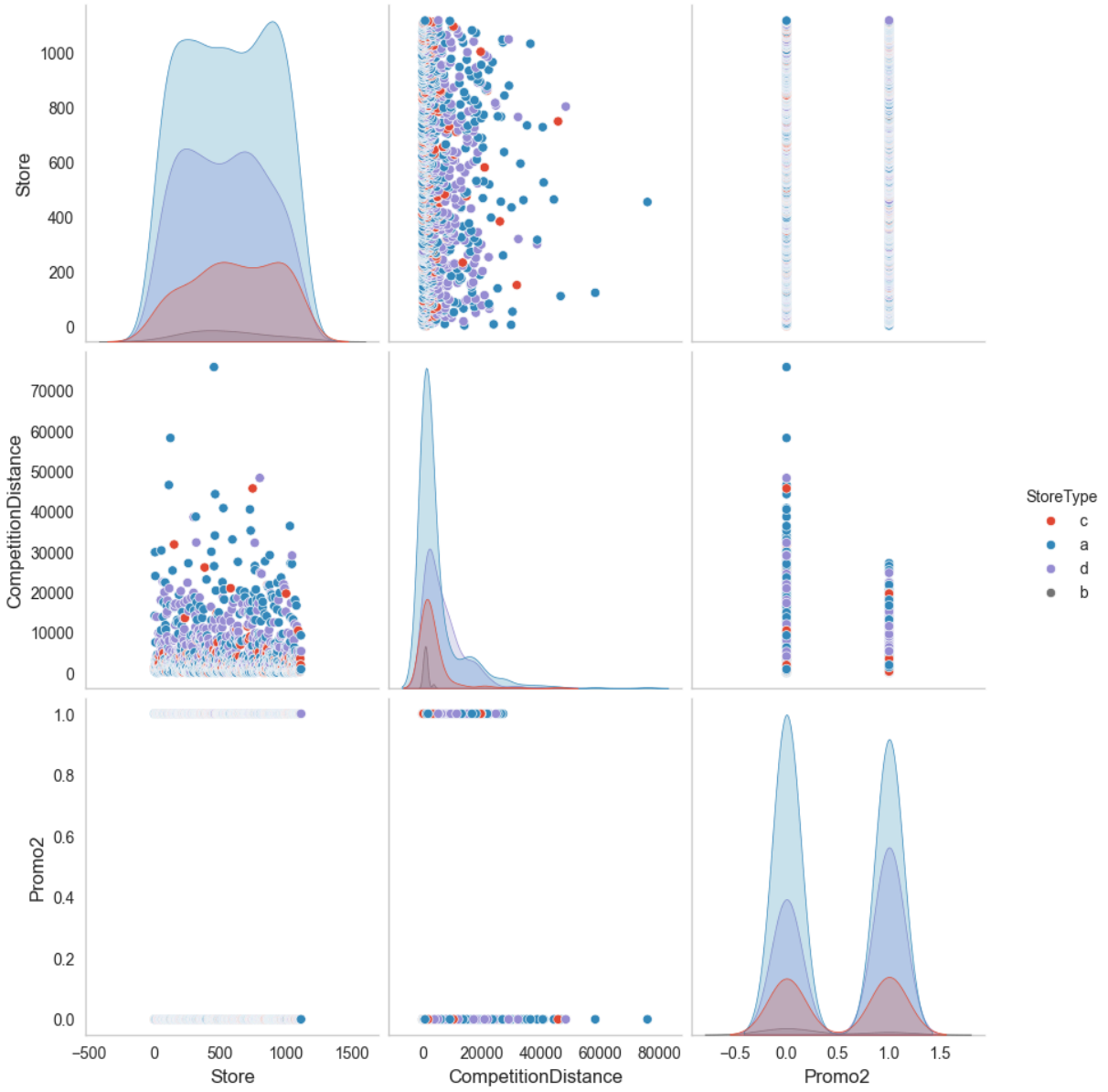
Out[24]: Text(0.5, 1.0, 'Distributin of Store Competition Distance')



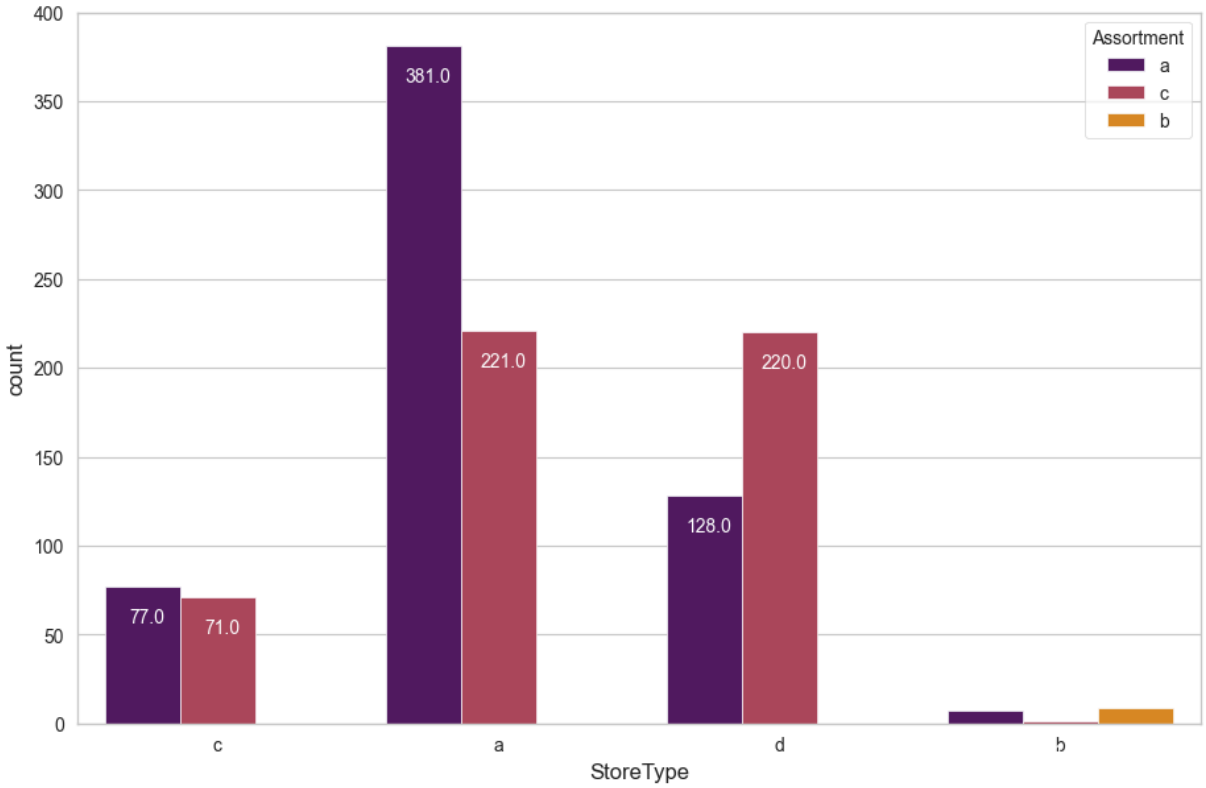
In [25]:
```
# replace missing values in CompetitionDistance with median for the store dataset

store_df.CompetitionDistance.fillna(store_df.CompetitionDistance.median(), inplace=
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_13208\2781003425.py:3: FutureWarning: A
value is trying to be set on a copy of a DataFrame or Series through chained assignm
ent using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform
the operation inplace on the original object.


  store_df.CompetitionDistance.fillna(store_df.CompetitionDistance.median(), inplace
=True)
```

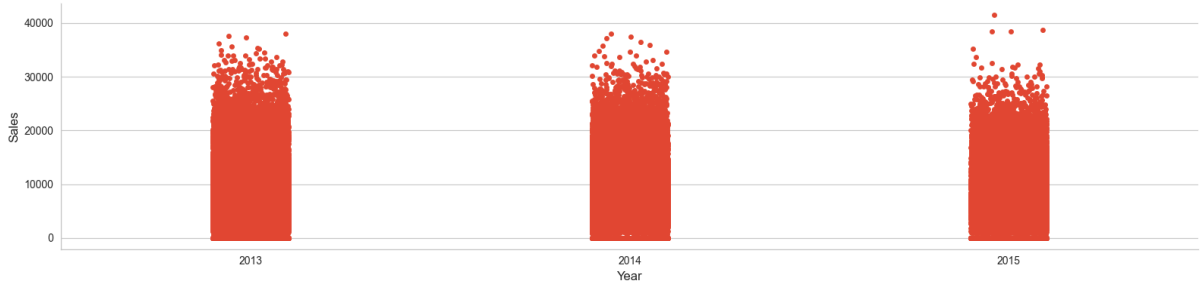
In [26]:
```
#pairplot for store dataset
sns.set_style("whitegrid", {'axes.grid' : False})
pp=sns.pairplot(store_df,hue='StoreType')
pp.fig.set_size_inches(10,10);
```



In [27]:
```
#checking stores with their assortment type
sns.set_style("whitegrid")
fig, ax = plt.subplots()
fig.set_size_inches(11, 7)
store_type=sns.countplot(x='StoreType',hue='Assortment', data=store_df,palette="inf

for p in store_type.patches:
    store_type.annotate(f'\n{p.get_height()}', (p.get_x()+0.15, p.get_height()),ha=
```



In [28]:
```
#plotting year vs sales
sns.catplot(x='Year',y='Sales',data=rossman_df, height=4, aspect=4 );
```



Merge Two Datasets

In [29]:
```
df = pd.merge(rossman_df, store_df, how='left', on='Store')
df.head()
```

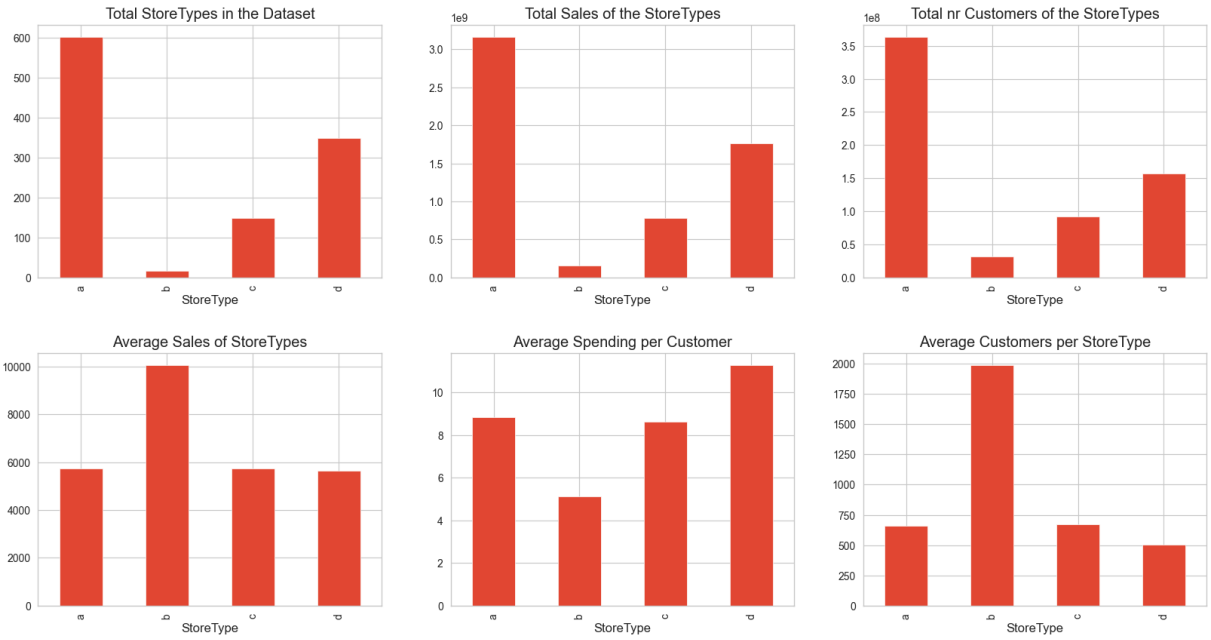
Out[29]:

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	SchoolHoliday	Year	Mon
0	1	5	2015-07-31	5263	555	1	1	1	2015	
1	2	5	2015-07-31	6064	625	1	1	1	2015	
2	3	5	2015-07-31	8314	821	1	1	1	2015	
3	4	5	2015-07-31	13995	1498	1	1	1	2015	
4	5	5	2015-07-31	4822	559	1	1	1	2015	

Data Analysis on Merge Dataset

In [30]:
```
df["Avg_Customer_Sales"] = df.Sales/df.Customers
```

In [31]:
```
f, ax = plt.subplots(2, 3, figsize = (20,10))

store_df.groupby("StoreType")["Store"].count().plot(kind = "bar", ax = ax[0, 0], ti
df.groupby("StoreType")["Sales"].sum().plot(kind = "bar", ax = ax[0,1], title = "To
df.groupby("StoreType")["Customers"].sum().plot(kind = "bar", ax = ax[0,2], title =
df.groupby("StoreType")["Sales"].mean().plot(kind = "bar", ax = ax[1,0], title = "A
df.groupby("StoreType")["Avg_Customer_Sales"].mean().plot(kind = "bar", ax = ax[1,1
df.groupby("StoreType")["Customers"].mean().plot(kind = "bar", ax = ax[1,2], title

plt.subplots_adjust(hspace = 0.3)
plt.show()
```

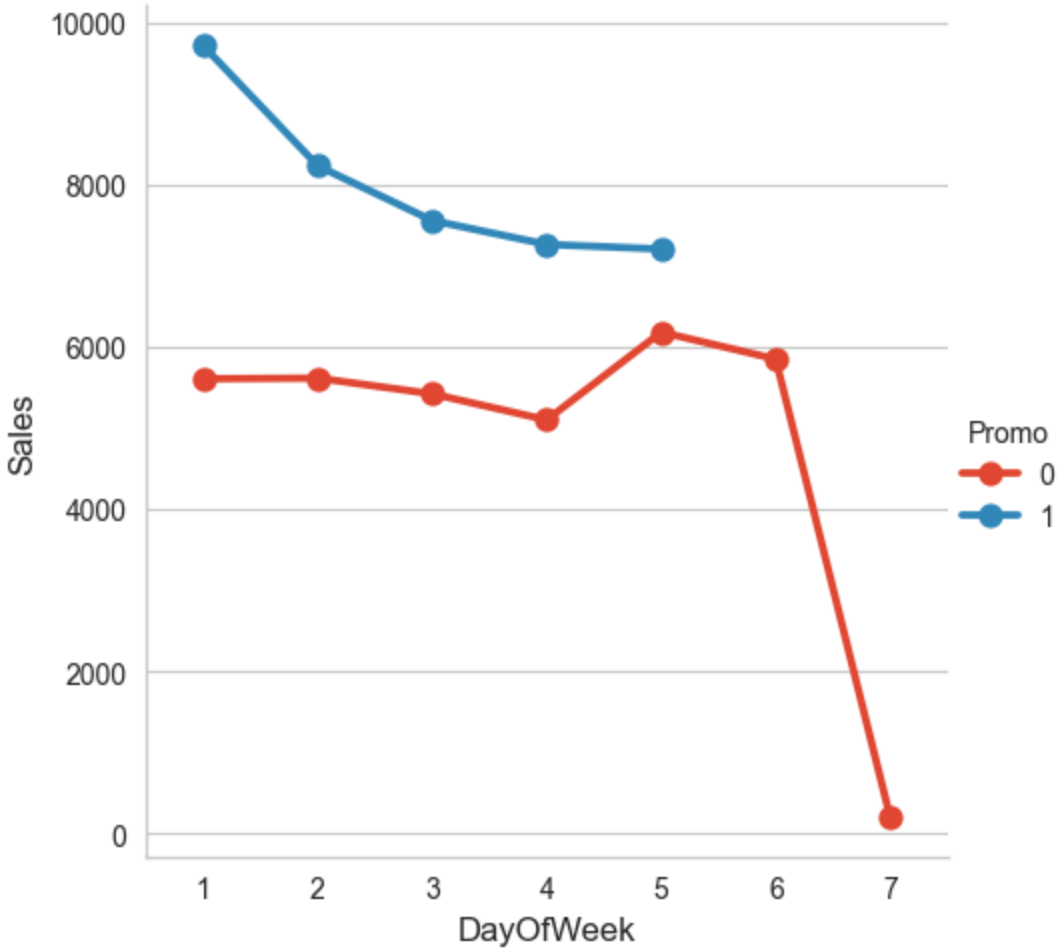


In [32]:
```
sns.catplot(data = df, x ="Month", y = "Sales",
                col = 'Promo', # per store type in cols
                hue = 'Promo2',
                row = "Year",
                kind="point"
               )
```

Out[32]: <seaborn.axisgrid.FacetGrid at 0x2888eafc560>

In [33]:
```
sns.catplot(data = df, x = "DayOfWeek", y = "Sales", hue = "Promo", kind="point")
```

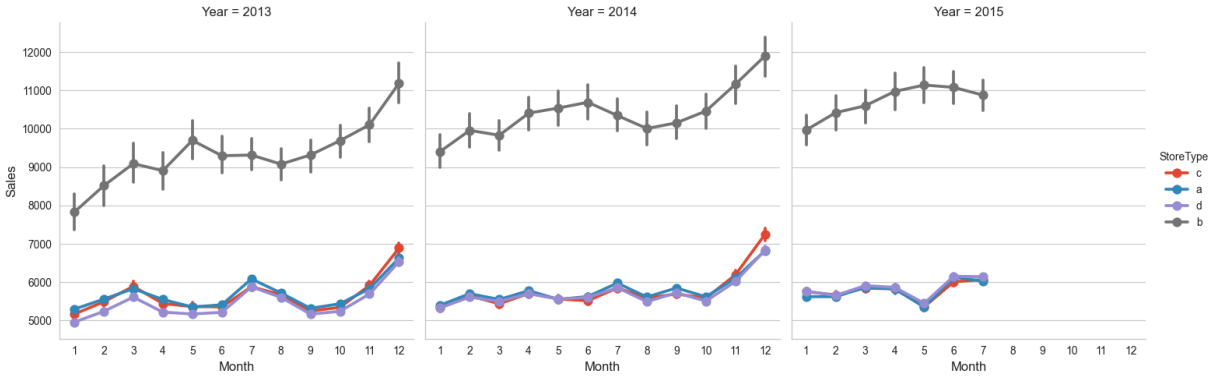
Out[33]:
```
<seaborn.axisgrid.FacetGrid at 0x2888b3edb80>
```



Check Seasonality

In [34]:
```
sns.catplot(data = df, x = "Month", y = "Sales", col = "Year", hue = "StoreType", k
```

Out[34]: <seaborn.axisgrid.FacetGrid at 0x288a871fef0>



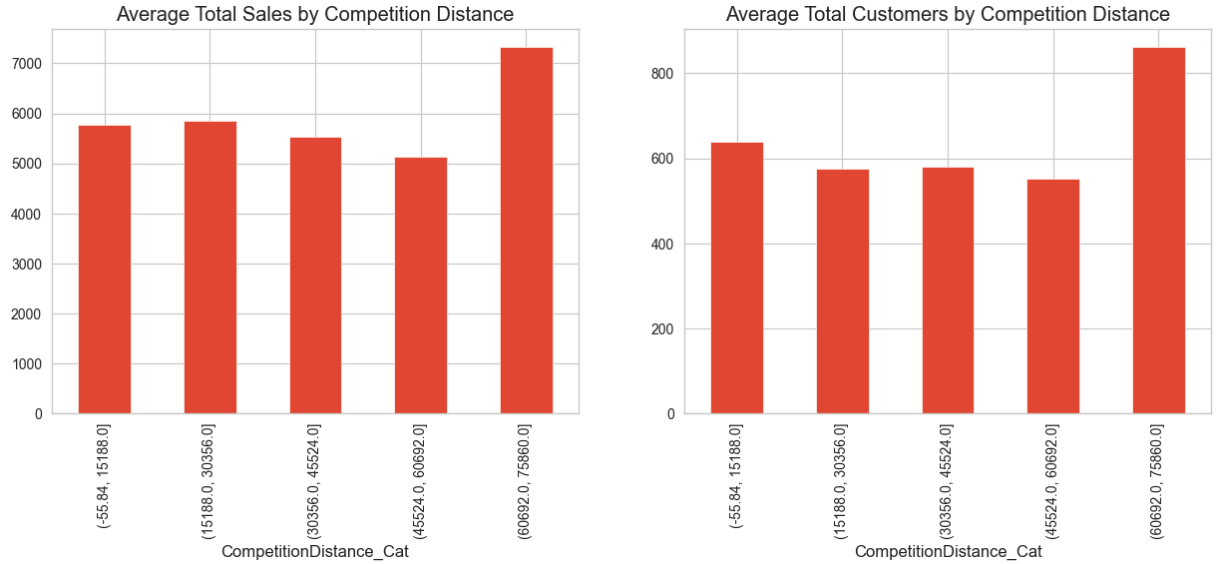
In [35]:
```
df.CompetitionDistance.describe()
# The obsverations are continous numbers, so we need to convert them into a categor
df["CompetitionDistance_Cat"] = pd.cut(df["CompetitionDistance"], 5)
```

In [36]:
```
f, ax = plt.subplots(1,2, figsize = (15,5))
```

```
df.groupby(by = "CompetitionDistance_Cat").Sales.mean().plot(kind = "bar", title =
df.groupby(by = "CompetitionDistance_Cat").Customers.mean().plot(kind = "bar", titl
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_13208\240981347.py:3: FutureWarning: The
default of observed=False is deprecated and will be changed to True in a future vers
ion of pandas. Pass observed=False to retain current behavior or observed=True to ad
opt the future default and silence this warning.
  df.groupby(by = "CompetitionDistance_Cat").Sales.mean().plot(kind = "bar", title =
"Average Total Sales by Competition Distance", ax = ax[0])
C:\Users\Admin\AppData\Local\Temp\ipykernel_13208\240981347.py:4: FutureWarning: The
default of observed=False is deprecated and will be changed to True in a future vers
ion of pandas. Pass observed=False to retain current behavior or observed=True to ad
opt the future default and silence this warning.
  df.groupby(by = "CompetitionDistance_Cat").Customers.mean().plot(kind = "bar", tit
le = "Average Total Customers by Competition Distance", ax = ax[1])
```

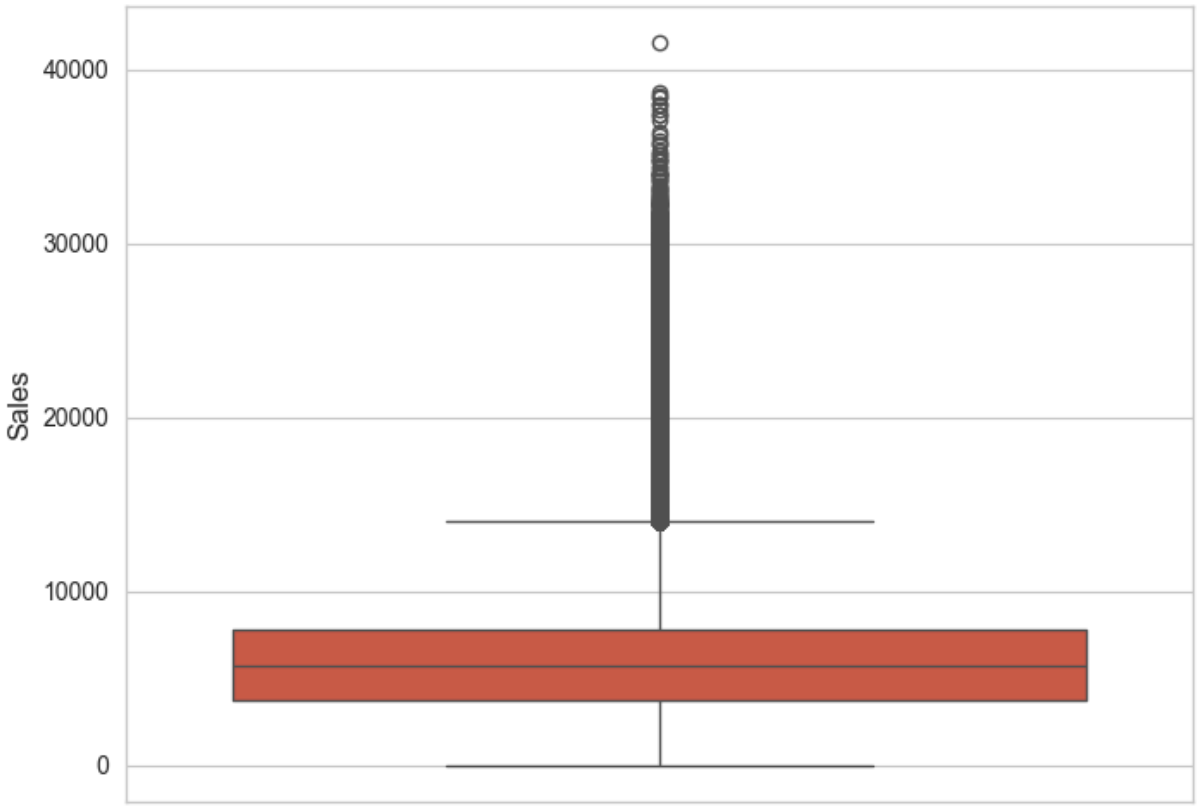
Out[36]: <Axes: title={'center': 'Average Total Customers by Competition Distance'}, xlabel ='CompetitionDistance_Cat'>



In [37]:
```
df.drop(['Avg_Customer_Sales','CompetitionDistance_Cat'],axis=1,inplace=True)
```

In [38]:
```
#checking outliers in sales
sns.boxplot(rossman_df['Sales'])
```

Out[38]: <Axes: ylabel='Sales'>



In [39]:
```
#removing outliers
def remove_outlier(df_in, col_name):
    q1 = df_in[col_name].quantile(0.25)
    q3 = df_in[col_name].quantile(0.75)
    iqr = q3-q1 #Interquartile range
    fence_low  = q1-1.5*iqr
    fence_high = q3+1.5*iqr
    df_out = df_in.loc[(df_in[col_name] > fence_low) & (df_in[col_name] < fence_hig
    return df_out
```

In [40]:
```
# defining new variable after removing outliers
df= remove_outlier(df, 'Sales')
```

Sales are highly correlated to number of Customers.

The most selling and crowded store type is A.

StoreType B has the lowest Average Sales per Customer. So i think customers visit this type only for small things.

StoreTybe D had the highest buyer cart.

Promo runs only in weekdays.

For all stores, Promotion leads to increase in Sales and Customers both.

More stores are opened during School holidays than State holidays.

The stores which are opened during School Holiday have more sales than normal days.

Sales are increased during Chirstmas week, this might be due to the fact that people buy more beauty products during a Christmas celebration.

Promo2 doesnt seems to be correlated to any significant change in the sales amount.

Absence of values in features CompetitionOpenSinceYear/Month doesn't indicate the absence of competition as CompetitionDistance values are not null where the other two values are null.

Drop Subsets Of Data Where Might Cause Bias

In [41]:
```
# where stores are closed, they won't generate sales, so we will remove that part o
df = df[df.Open != 0]
```

In [42]:
```
# Open isn't a variable anymore, so we'll drop it too
df = df.drop('Open', axis=1)
```

In [43]:
```
# Check if there's any opened store with zero sales
df[df.Sales == 0]['Store'].sum()
```

Out[43]:
```
np.int64(31460)
```

In [44]:
```
# see the percentage of open stored with zero sales
df[df.Sales == 0]['Sales'].sum()/df.Sales.sum()
```

Out[44]:
```
np.float64(0.0)
```

In [45]:
```
# remove this part of data to avoid bias
df = df[df.Sales != 0]
```

In [46]:
```
df_new=df.copy()
```

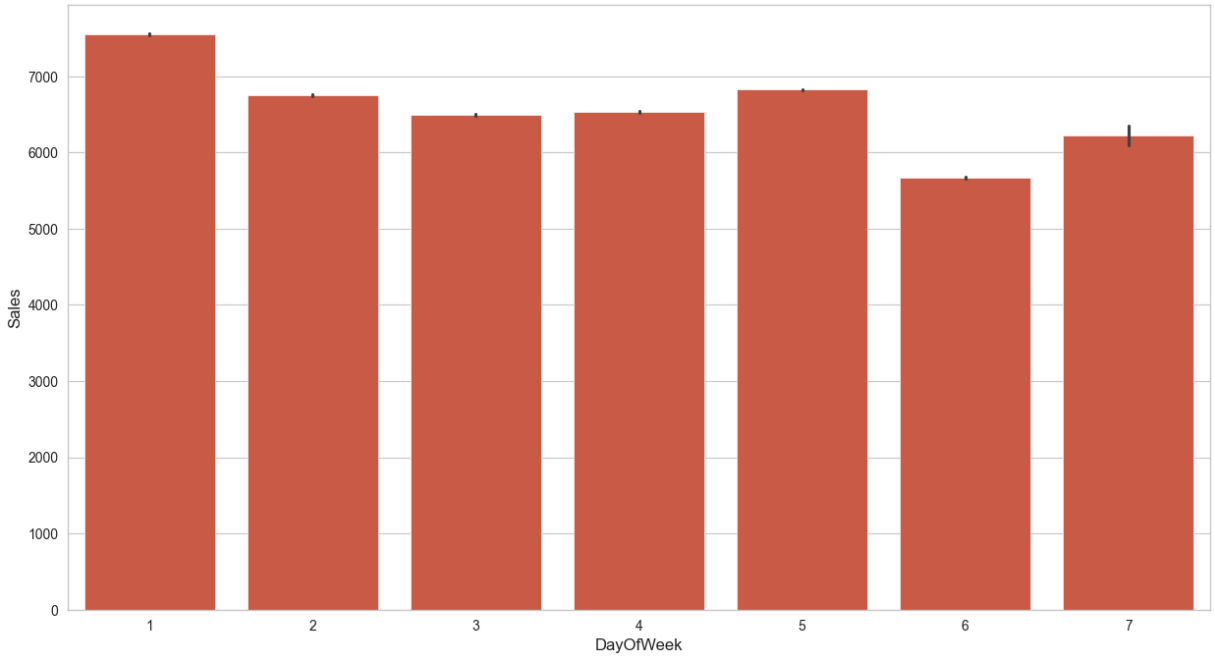
In [47]:
```
df_new = pd.get_dummies(df_new,columns=['StoreType','Assortment'], dtype=int)
```

In [48]:
```
df_new.head()
```

Out[48]:

	Store	DayOfWeek	Date	Sales	Customers	Promo	SchoolHoliday	Year	Month	Da
0	1	5	2015-07-31	5263	555	1	1	2015	7	3
1	2	5	2015-07-31	6064	625	1	1	2015	7	3
2	3	5	2015-07-31	8314	821	1	1	2015	7	3
3	4	5	2015-07-31	13995	1498	1	1	2015	7	3
4	5	5	2015-07-31	4822	559	1	1	2015	7	3

In [49]:
```
#plot for sales in terms of days ofthe week
plt.figure(figsize=(15,8))
sns.barplot(x='DayOfWeek', y='Sales' ,data=df_new);
```



Set Feature/Target Variable

In [50]:
```
X = df_new.drop(['Sales','Store','Date','Year'] , axis = 1)
y= df_new.Sales
```

In [51]:
```
X.shape
```

Out[51]: (817644, 16)

In [52]:
```
X.head()
```

Out[52]:

	DayOfWeek	Customers	Promo	SchoolHoliday	Month	Day	WeekOfYear	Competition
0	5	555	1	1	7	31	31	
1	5	625	1	1	7	31	31	
2	5	821	1	1	7	31	31	
3	5	1498	1	1	7	31	31	
4	5	559	1	1	7	31	31	

In [53]:
```
y.head()
```

Out[53]:
```
0     5263
1     6064
2     8314
3    13995
4     4822
Name: Sales, dtype: int64
```

Splitting Dataset Into Training Set and Test Set

In [54]:
```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3, random_state=0)
```

In [55]:
```
columns=X_train.columns
```

Different options of Supervised Machine Learning Algorithms

1. Linear Regression (OLS)

In [56]:
```
# Transforming data
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [57]:
```
# Fitting Multiple Linear Regression to the Training set
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

Out[57]:
```
LinearRegression
LinearRegression()
```

In [58]:
```
regressor.intercept_
```

Out[58]:
```
np.float64(-1507.4417985697683)
```

In [59]:
```
regressor.coef_
```

Out[59]:
```
array([-1.30381346e+02,  3.06863041e+04,  1.09386001e+03,  2.29418388e+01,
        3.54223020e+02,  3.82831239e+01, -1.58021936e+02,  1.82430640e+03,
        3.10732033e+02,  3.18132398e+02, -1.90167603e+03,  1.67419398e+02,
        1.41612423e+03,  1.44688318e+03, -3.17950648e+03,  1.73262330e+03])
```

In [60]:
```
y_pred_train = regressor.predict(X_train)
```

In [61]:
```
# Predicting the Test set results
y_pred = regressor.predict(X_test)
```

In [62]:
```
mean_squared_error(y_test, y_pred)
```

Out[62]:
```
1329406.2402058858
```

In [63]:
```
# Test performance
math.sqrt(mean_squared_error(y_test, y_pred))
```

Out[63]:
```
1152.9988032109513
```

In [64]:
```
train_score_1=regressor.score(X_train,y_train)
train_score_1
```

Out[64]:
```
0.7807496727472851
```

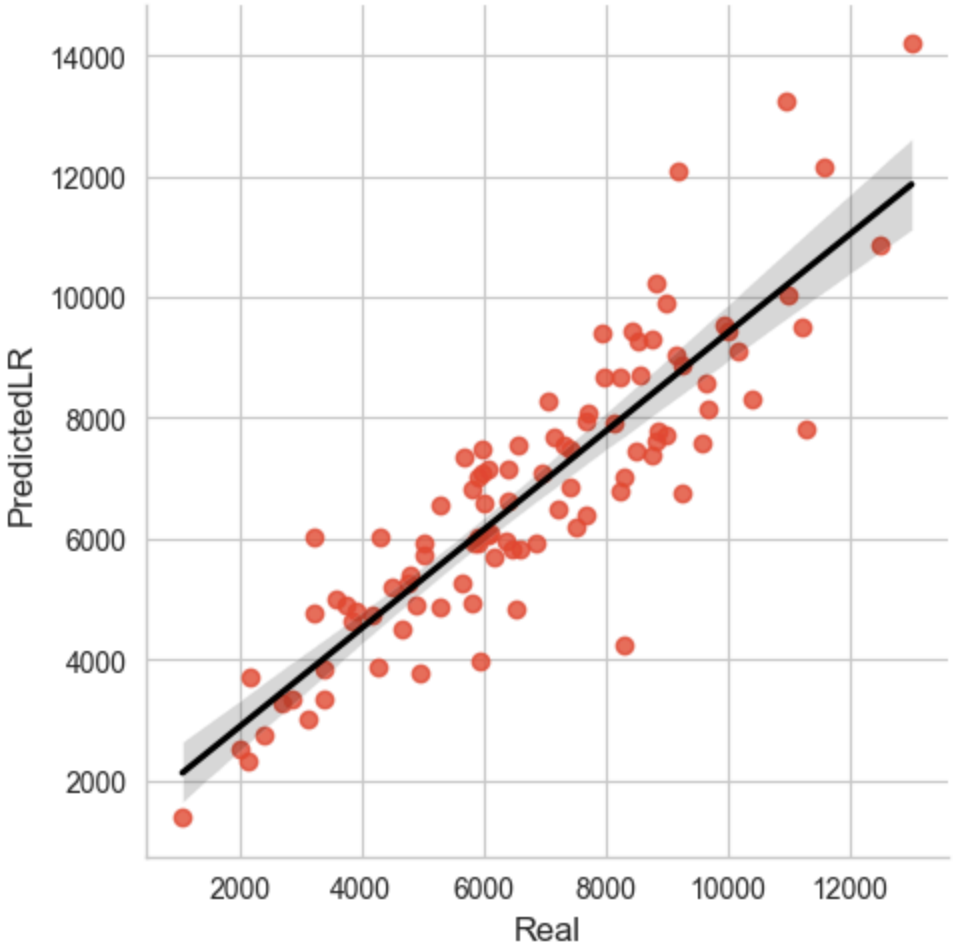
In [65]:
```
test_score_1=regressor.score(X_test,y_test)
test_score_1
```

Out[65]:
```
0.7823919595957616
```

In [66]:
```
#storing 100 observations for analysis
simple_lr_pred = y_pred[:100]
simple_lr_real = y_test[:100]
dataset_lr = pd.DataFrame({'Real':simple_lr_real,'PredictedLR':simple_lr_pred}) #st
```

In [67]:
```
#storing absolute diffrences between actual sales price and predicted
dataset_lr['diff']=(dataset_lr['Real']-dataset_lr['PredictedLR']).abs()
```

In [68]:
```
#visualising our predictions
sns.lmplot(x='Real', y='PredictedLR', data=dataset_lr, line_kws={'color': 'black'})
```



Inferences on OLS

In [69]:
```
X = sm.add_constant(X) ## let's add an intercept (beta_0) to our model
model = sm.OLS(y, X).fit() ## sm.OLS(output, input)
predictions = model.predict(X)

# Print out the statistics
model.summary()
```

Out[69]:

OLS Regression Results

Dep. Variable:	Sales	**R-squared:**	0.781
Model:	OLS	**Adj. R-squared:**	0.781
Method:	Least Squares	**F-statistic:**	2.086e+05
Date:	Tue, 04 Mar 2025	**Prob (F-statistic):**	0.00
Time:	15:45:03	**Log-Likelihood:**	-6.9257e+06
No. Observations:	817644	**AIC:**	1.385e+07
Df Residuals:	817629	**BIC:**	1.385e+07
Df Model:	14		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	-993.1075	5.669	-175.169	0.000	-1004.219	-981.996
DayOfWeek	-21.0704	0.785	-26.837	0.000	-22.609	-19.532
Customers	7.2260	0.005	1471.966	0.000	7.216	7.236
Promo	1093.4894	2.768	395.028	0.000	1088.064	1098.915
SchoolHoliday	27.2192	3.326	8.183	0.000	20.700	33.738
Month	33.1415	1.433	23.134	0.000	30.334	35.949
Day	1.2777	0.152	8.418	0.000	0.980	1.575
WeekOfYear	-3.3867	0.330	-10.267	0.000	-4.033	-2.740
CompetitionDistance	0.0240	0.000	140.522	0.000	0.024	0.024
Promo2	307.7765	2.662	115.635	0.000	302.560	312.993
StoreType_a	66.5792	4.780	13.929	0.000	57.211	75.948
StoreType_b	-2140.7862	12.728	-168.195	0.000	-2165.733	-2115.840
StoreType_c	-81.9285	5.313	-15.419	0.000	-92.342	-71.514
StoreType_d	1163.0279	5.024	231.517	0.000	1153.182	1172.874
Assortment_a	1114.6637	6.456	172.654	0.000	1102.010	1127.317
Assortment_b	-3510.5496	15.385	-228.183	0.000	-3540.703	-3480.396
Assortment_c	1402.7783	6.642	211.194	0.000	1389.760	1415.797

Omnibus:	49108.747	**Durbin-Watson:**	1.716
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	145480.294
Skew:	0.301	**Prob(JB):**	0.00

Kurtosis:	4.977	**Cond. No.**	8.10e+18

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 1.14e-24. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.

Decision Tree

In [70]:
```
tree = DecisionTreeRegressor()
treereg = tree.fit(X_train, y_train)
```

In [71]:
```
def rmse(x, y):
    return sqrt(mean_squared_error(x, y))

# definte MAPE function
def mape(x, y):
    return np.mean(np.abs((x - y) / x)) * 100


train_score_3=treereg.score(X_train, y_train)
test_score_3=treereg.score(X_test, y_test)

print("Regresion Model Score" , ":" , train_score_3 , "," ,
      "Test Score" ,":" , test_score_3)

y_predicted = treereg.predict(X_train)
y_test_predicted = treereg.predict(X_test)
print("Training RMSE", ":", rmse(y_train, y_predicted),
      "Testing RMSE", ":", rmse(y_test, y_test_predicted))
print("Training MAPE", ":", mape(y_train, y_predicted),
      "Testing MAPE", ":", mape(y_test, y_test_predicted))
```

```
Regresion Model Score : 0.9999957301266714 , Test Score : 0.9155407754195621
Training RMSE : 5.097176662035323 Testing RMSE : 718.3149947722069
Training MAPE : 0.0010365300881590655 Testing MAPE : 7.643891032718446
```

K Nearest Neighbours

In [72]:
```
from sklearn.neighbors import KNeighborsRegressor
knn = KNeighborsRegressor(n_neighbors = 30)
knnreg = knn.fit(X_train, y_train)
```

In [73]:
```
def rmse(x, y):
    return sqrt(mean_squared_error(x, y))

# definte MAPE function
def mape(x, y):
    return np.mean(np.abs((x - y) / x)) * 100

print("Regresion Model Score" , ":" , knnreg.score(X_train, y_train) , "," ,
```

```
      "Out of Sample Test Score" ,":" , knnreg.score(X_test, y_test))

y_predicted = knnreg.predict(X_train)
y_test_predicted = knnreg.predict(X_test)

print("Training RMSE", ":", rmse(y_train, y_predicted),
      "Testing RMSE", ":", rmse(y_test, y_test_predicted))
print("Training MAPE", ":", mape(y_train, y_predicted),
      "Testing MAPE", ":", mape(y_test, y_test_predicted))
```

```
Regresion Model Score : 0.7372261980067728 , Out of Sample Test Score : 0.7166584142
323265
Training RMSE : 1264.4836631252208 Testing RMSE : 1315.6687141978773
Training MAPE : 16.244493588517088 Testing MAPE : 16.928417525093515
```

Random Forest

In [74]:
```
rdf = RandomForestRegressor(n_estimators=80,min_samples_split=2, min_samples_leaf=1
rdfreg = rdf.fit(X_train, y_train)
```

In [75]:
```
def rmse(x, y):
    return sqrt(mean_squared_error(x, y))

# definte MAPE function
def mape(x, y):
    return np.mean(np.abs((x - y) / x)) * 100

train_score_5=rdfreg.score(X_train, y_train)
test_score_5=rdfreg.score(X_test, y_test)

print("Regresion Model Score" , ":" , train_score_5 , "," ,
      "Test Score" ,":" , test_score_5)

y_predicted_2 = rdfreg.predict(X_train)
y_test_predicted_2 = rdfreg.predict(X_test)

print("Training RMSE", ":", rmse(y_train, y_predicted_2),
      "Testing RMSE", ":", rmse(y_test, y_test_predicted_2))
print("Training MAPE", ":", mape(y_train, y_predicted_2),
      "Testing MAPE", ":", mape(y_test, y_test_predicted_2))
```

```
Regresion Model Score : 0.9937773262338249 , Test Score : 0.9565097583110082
Training RMSE : 194.58560402208258 Testing RMSE : 515.4508950151943
Training MAPE : 2.1119079735431883 Testing MAPE : 5.658234750983677
```

In [ ]: